

Applying Supervised Learning to Predict Student Dropout

Mahmood Ahmad, PhD

November 7, 2024

Contents

1	Introduction and Overview	3
2	Correlation Analysis of Numerical Features	3
2.1	Exploratory Analysis Including Additional Features	3
3	Neural Network (NN) Modelling	10
3.1	Modelling with Original Data with Dropped Features	10
3.1.1	NN Models without Hyperparameter Tuning	10
3.1.2	NN Models with Hyperparameter Tuning	12
3.2	Modelling Data with Additional Features	12
3.2.1	NN Models without Hyperparameter Tuning	12
3.2.2	NN Models with Hyperparameter Tuning	17
4	Modelling with XGBoost Algorithm	21
4.1	Modelling the Original Data with Dropped Features	21
4.1.1	XGBoost without Hyperparameter Tuning	21
4.1.2	XGBoost with Hyperparameter Tuning	21
4.2	Modelling Data with Additional Features	23
4.2.1	XGBoost without Hyperparameter Tuning	23
4.2.2	XGBoost with Hyperparameter Tuning	23
4.2.3	Feature Importance	23
5	Conclusions	27

1 Introduction and Overview

The student data-set has 36 features with 25059 rows. Deleting all the rows with null values leaves only 176 rows in dataset. Therefore, we decided to drop the columns which the project guideline has advised us before removing the rows with null values, resulting in 22686 columns with 9 features. We implemented One-Hot Encoding (OHE) to the categorical features such that we did not group their values with smaller counts so that the effects of minor observations is not lost during training. Binary encoding was applicable on a few features, detailed in code. After the encoding, the dataset contained 68 features with three original numerical features.

The feature `DropOut`, which was the logical inverse of `CompletedCourse` was the target variable.

2 Correlation Analysis of Numerical Features

There was a considerable negative correlation (Figure 1) between `CreditWeightedAverage` and `DropOut`, and also with `UnauthorisedAbsenceCount`. Students with higher credit weighted average will obviously be less probable to drop out. Similarly, a positive correlation (not as strong) between `UnauthorisedAbsenceCount` and the `DropOut` variables was contextually obvious.

The pair-plots with `DropOut` as hue (Figure 2) depict that the concentration of `DropOut` values is higher in the region with low `CreditWeightedAverage` score. There are some higher `UnauthorisedAbsenceCount` scores for younger ages for drop outs. The drop-outs are more concentrated for lower `CreditWeightedAverage` scores, for both lower and higher `UnauthorisedAbsenceCount` values.

In the box-plots of numerical features, grouped by `DropOut` (Figure 3), the `CreditWeightedAverage` is distributed with a very low median value (about -3) for those who who dropped out. The `UnauthorisedAbsenceCount` is also higher on average for drop-outs than those who completed the course.

2.1 Exploratory Analysis Including Additional Features

With the addition of `ContactHours` and `AttendancePercentage` features, our correlation analysis of data provided (Figures 4 and 5) significant correlations between features. The removal of rows with null values resulted in 22686 rows and 70 features after carrying out the necessary encodings.

The included feature `AttendancePercentage` has shown considerably strong correlation with `DropOut` (negative) and a positive correlation with `CreditWeightedAverage` (Figures 4 and 5), which is contextually obvious. The grouped box-

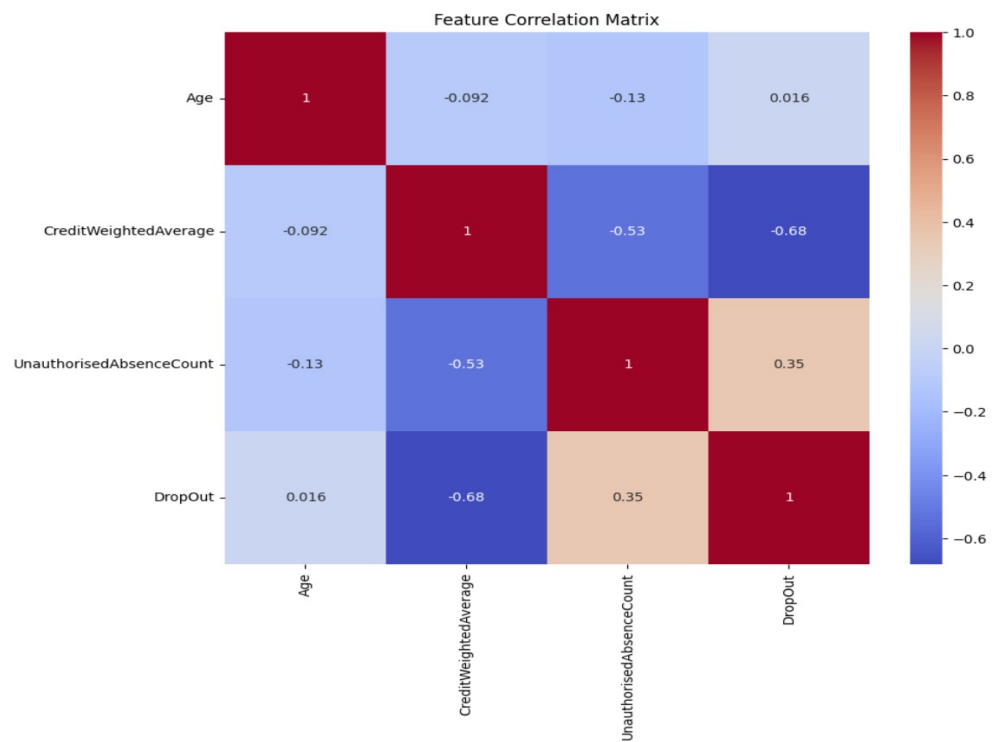


Figure 1: The **seaborn** heatmap depicting correlation between the numerical features.

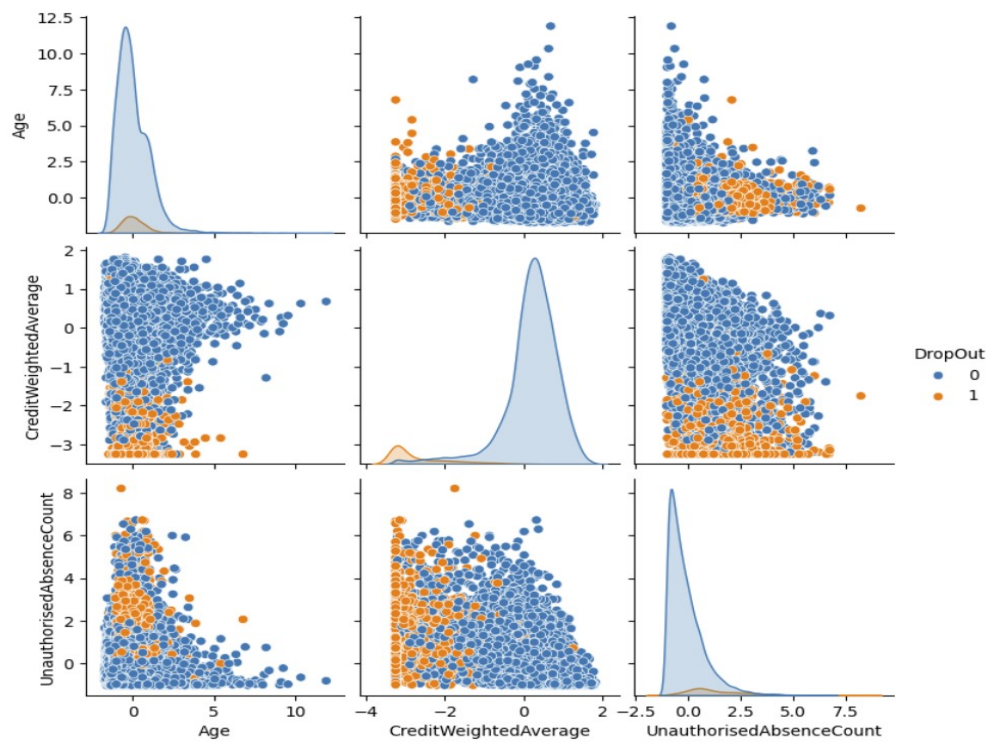


Figure 2: The seaborn pair-plots, with DropOut feature as hue, depicting correlation between the numerical features.

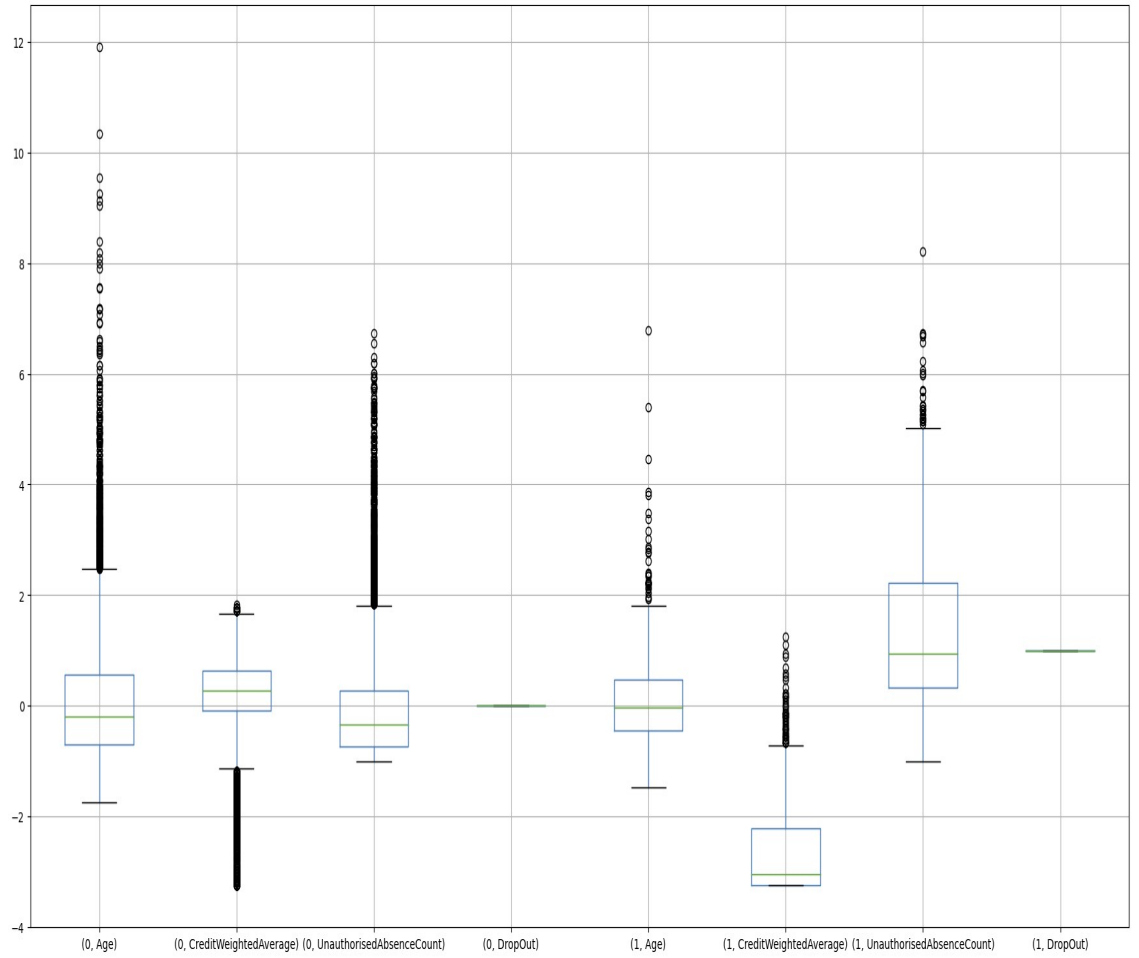


Figure 3: The boxplots of numerical features, grouped by DropOut feature.

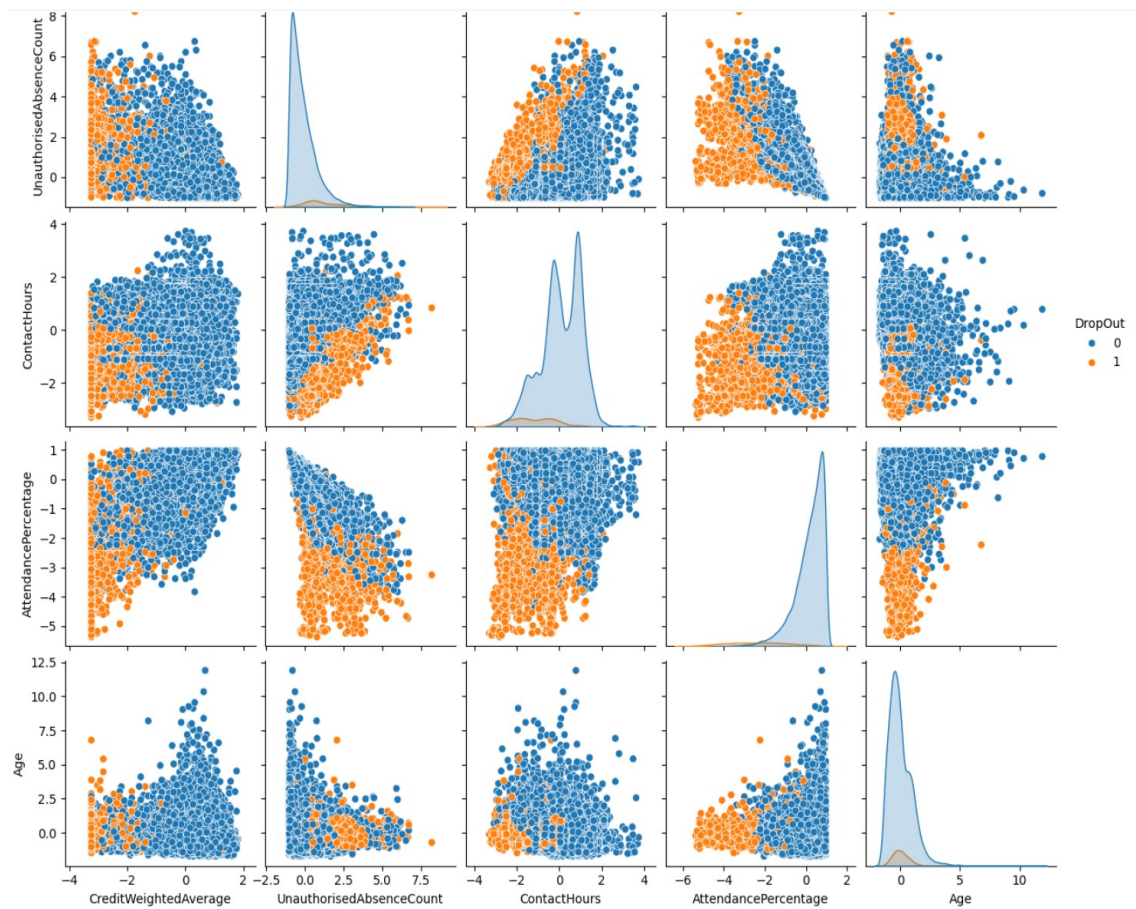


Figure 4: Pair-plots for students data including the additional features.

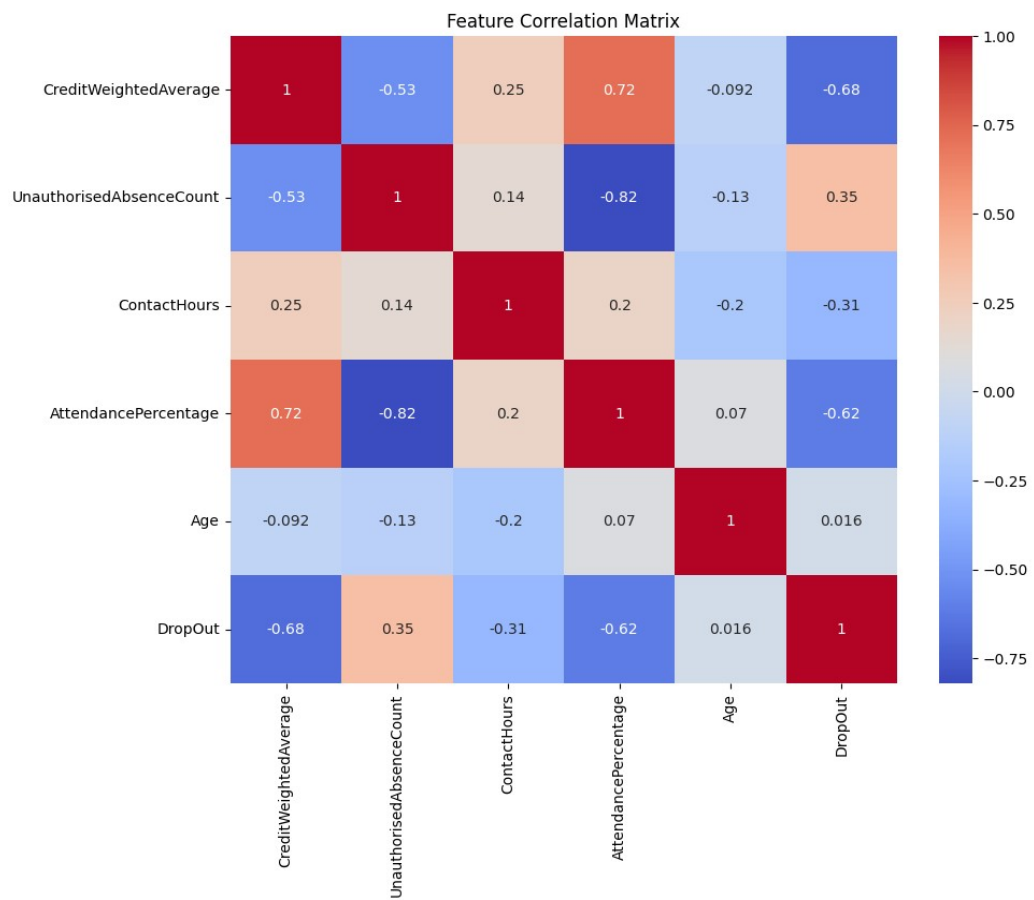


Figure 5: Correlation heatmap for students data including the additional features.

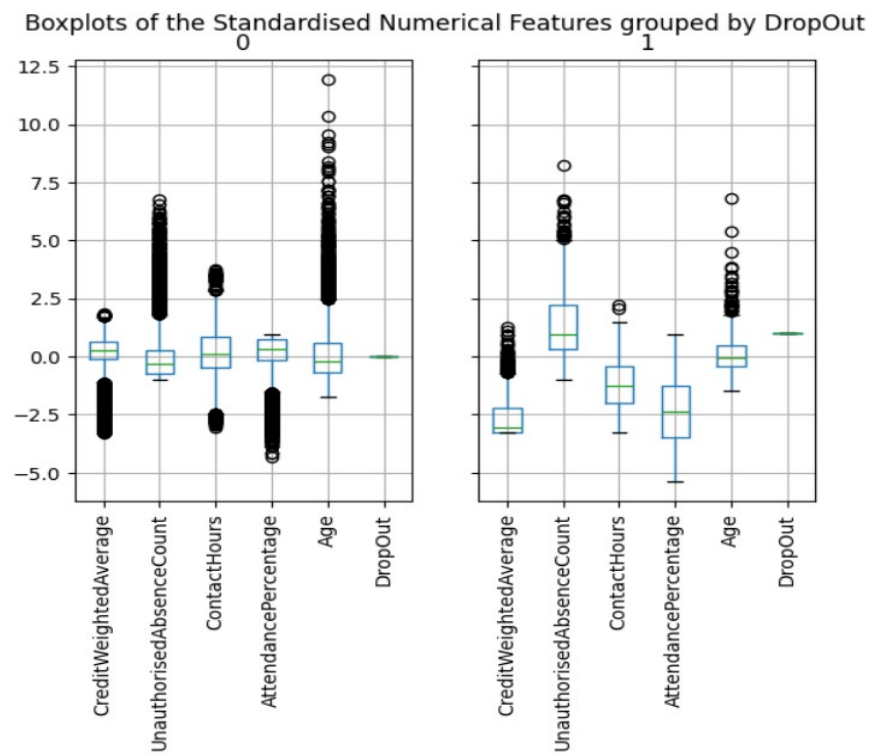


Figure 6: Box-plots of students data with additional features grouped by DropOut.

Model Name	Optimiser	Hidden Layers	Neurons per hidden layer	Activation Function	batch size	epochs
Model 1	Adam	2	64	Relu	16	10
Model 2	RMSProp	2	64	Relu	16	10

Table 1: Neural Network model parameters for initial experiments without hyperparameter tuning.

Model	train_acc	train_loss	test_acc	test_loss	valid_acc	valid_loss
Model 1	0.9739	0.0675	0.9722	0.0857	0.9642	0.0862
Model 2	0.9742	0.0821	0.9720	0.0857	0.9642	0.1160

Table 2: Accuracy and Loss metrics for the two NN models: Model 1 and Model 2.

plots of numerical features are shown in Figure 6. The correlation of these additional features indicates their significance for this study, which may be more evident with our experiments shortly.

3 Neural Network (NN) Modelling

3.1 Modelling with Original Data with Dropped Features

3.1.1 NN Models without Hyperparameter Tuning

Scaling of features was carried out after `train_test_split` of data set for the neural networks. Training parameters for models with Adam and RMSProp as optimisers are in Table 1. The accuracy and loss results for training, test and validation data are in Table 2. Model 1 has produced only a marginal improvement in accuracy over RMSProp-based Model 2, because Model 1 has relatively lower precision (more false positives) and bigger recall (more false negatives) than Model 2 (Figures 7 and 8, and Table 3).

Model	Optimiser	Accuracy	Precision	Recall	F1 Score
Model 1	Adam	0.9722	0.8029	0.7474	0.7742
Model 2	RMSProp	0.9720	0.8045	0.7405	0.7712

Table 3: Performance metrics for NN Model 1 and Model 2.

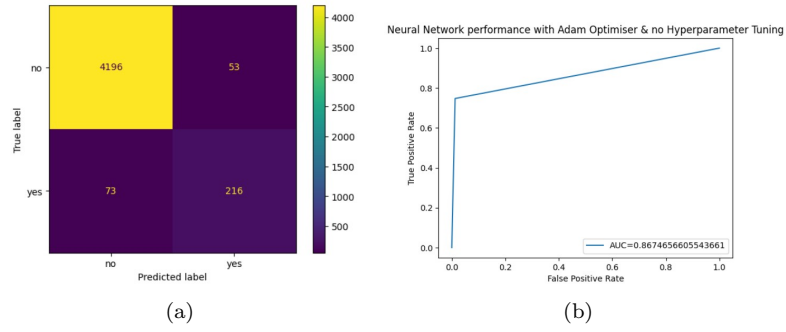


Figure 7: Model 1 - Adam optimiser: (a) Confusion Matrix; (b) ROC_AUC_Curve and AUC score.

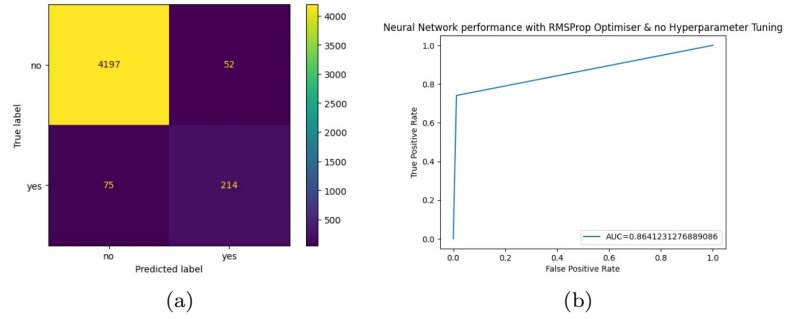


Figure 8: Model 2 - RMSProp optimiser: (a) Confusion Matrix; (b) ROC_AUC_Curve and AUC score.

Model Name	Hidden Layers	Neurons per layer	Activation fn. in hidden layers	batch_size	epochs	L2 Reg
Model 3	1	1	Relu	16	10	0.005
Model 4	1	1	Relu	16	10	0.01
Model 5	1	1	Relu	16	10	0.05
Model 6	1	32	Relu	16	10	0.005
Model 7	1	32	Relu	16	10	0.01
Model 8	1	32	Relu	16	10	0.05
Model 9	2	64	Relu	16	10	0.005
Model 10	2	64	Relu	16	10	0.01
Model 11	2	64	Relu	16	10	0.05
Model 12	3	128	Relu	16	10	0.005
Model 13	3	128	Relu	16	10	0.01
Model 14	3	128	Relu	16	10	0.05

Table 4: Neural Network model parameters with Adam Optimiser and hyperparametric tuning

3.1.2 NN Models with Hyperparameter Tuning

For hyperparameter tuning, we used varying values of **learning rate**, **L2 regularization** and **patience** = 1. Twelve different models each for the **Adam** and **RMSProp** optimisers with various configurations (Table 4, same for both optimisers) were trained. We analysed performance metrics for the two groups of models (Models 3 to 14, and Models 15 to 26) in Table 5 and 6 respectively, the accuracy and loss curves for **Adam**-based models in Figures 9 and 10, and those for **RMSProp**-based models in Figures 11 and 12. Method 15 produced a slightly improved accuracy of 97.4% over Model 9, with a better precision of 82% and a recall of 76% (Figure 13 for confusion matrix and the ROC-AUC curve). Thus, Model 15 was the best model among the NN models with hyperparameter tuning, albeit only a very small improvement over Model 1 and 2 (trained without hyperparametric tuning).

3.2 Modelling Data with Additional Features

3.2.1 NN Models without Hyperparameter Tuning

Models 27 and 28 were defined with model parameters given in Table 7, similar to Table 1. Performance metrics (Table 8) for these two models highlight the positive impact of these additional features. The **Adam**-based model has a very slight improvement over **RMSProp**-based model, with better precision and lower recall (see Figures 14 and 15).

Model	Optimiser	Accuracy	Precision	Recall	F1 Score
Model 3	Adam	0.971794	0.790614	0.757785	0.773852
Model 4	Adam	0.936316	0.000000	0.000000	0.000000
Model 5	Adam	0.968929	0.764286	0.740484	0.752197
Model 6	Adam	0.970912	0.762542	0.788927	0.775510
Model 7	Adam	0.971573	0.766667	0.795848	0.780985
Model 8	Adam	0.970031	0.800000	0.705882	0.750000
Model 9	Adam	0.973116	0.801444	0.768166	0.784452
Model 10	Adam	0.966064	0.787234	0.640138	0.706107
Model 11	Adam	0.969149	0.767025	0.740484	0.753521
Model 12	Adam	0.972014	0.818898	0.719723	0.766114
Model 13	Adam	0.972234	0.774411	0.795848	0.784983
Model 14	Adam	0.972014	0.757962	0.823529	0.789386

Table 5: Performance metrics for Adam optimiser-based models.

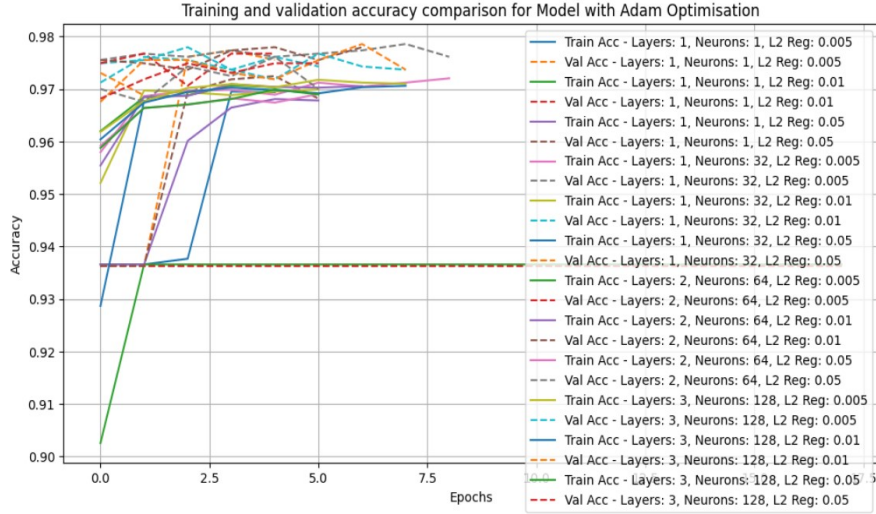


Figure 9: Training and Validation accuracy curves for various models with Adam Optimiser and hyperparametric tuning.

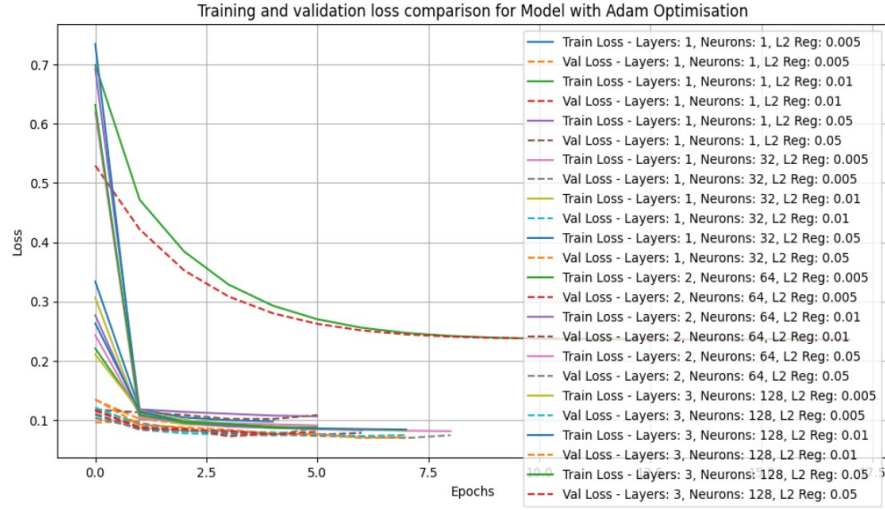


Figure 10: Training and Validation loss curves for various models with Adam Optimiser and hyperparametric tuning.

Model	Optimiser	Accuracy	Precision	Recall	F1 Score
Model 15	RMSProp	0.974438	0.821561	0.764706	0.792115
Model 16	RMSProp	0.970912	0.777385	0.761246	0.769231
Model 17	RMSProp	0.969370	0.758621	0.761246	0.759931
Model 18	RMSProp	0.970912	0.783394	0.750865	0.766784
Model 19	RMSProp	0.970912	0.759076	0.795848	0.777027
Model 20	RMSProp	0.966946	0.732441	0.757785	0.744898
Model 21	RMSProp	0.971133	0.758170	0.802768	0.779832
Model 22	RMSProp	0.971133	0.776224	0.768166	0.772174
Model 23	RMSProp	0.967827	0.809524	0.647059	0.719231
Model 24	RMSProp	0.970031	0.759322	0.775087	0.767123
Model 25	RMSProp	0.967607	0.800847	0.653979	0.720000
Model 26	RMSProp	0.969590	0.747541	0.788927	0.767677

Table 6: Performance metrics for RMSProp-based models with hyperparametric tuning.

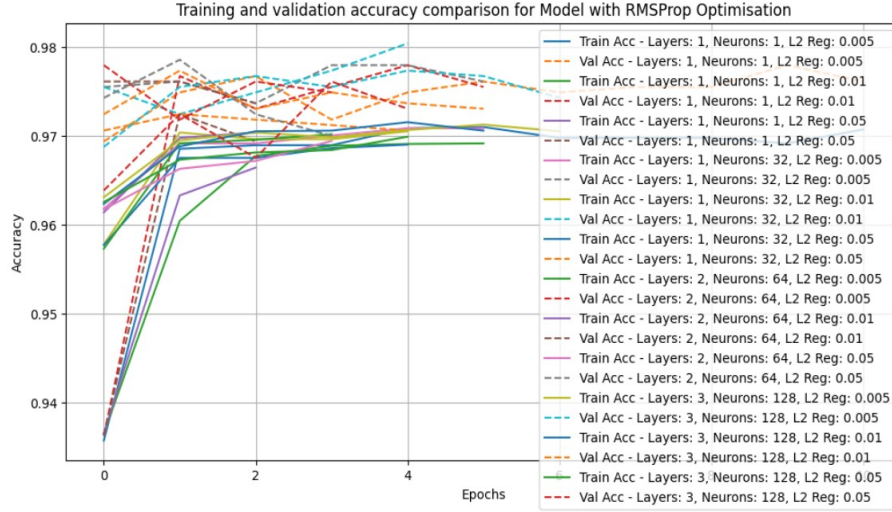


Figure 11: Training and Validation accuracy curves for various models with RMSProp Optimiser and hyperparametric tuning.

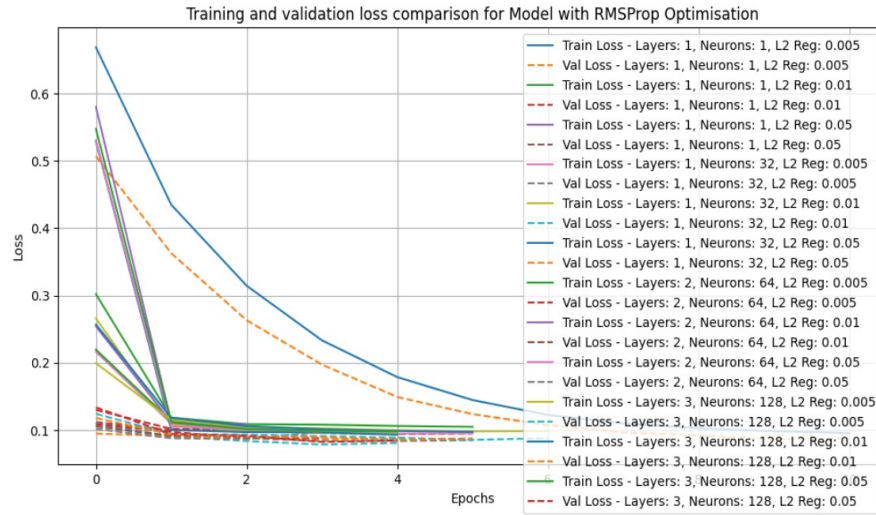


Figure 12: Training and Validation loss curves for various models with RMSProp Optimiser and hyperparametric tuning.

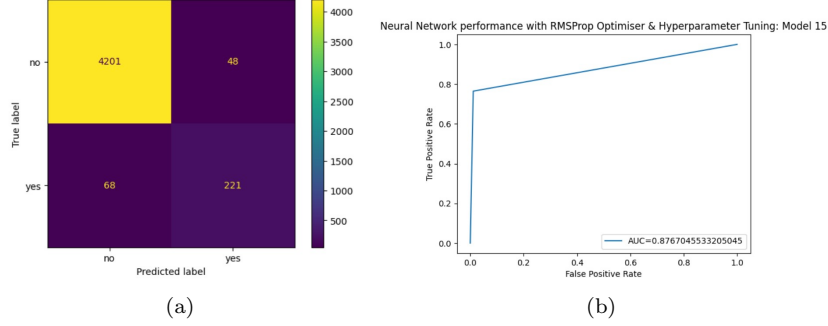


Figure 13: Model 15: (a) Confusion Matrix; (b) ROC_AUC_Curve and AUC score.

Model Name	Optimiser	Hidden Layers	Neurons per hid-den layer	Activation Function	batch size	epochs
Model 27	Adam	2	64	Relu	16	10
Model 28	RMSProp	2	64	Relu	16	10

Table 7: Neural Network model parameters for initial experiments with additional features.

Model	Optimiser	Accuracy	Precision	Recall	F1 Score
Model 27	Adam	0.979066	0.870229	0.788927	0.827586
Model 28	RMSProp	0.978845	0.838596	0.826990	0.832753

Table 8: Performance metrics for NN Models 27 and 28, with additional features.

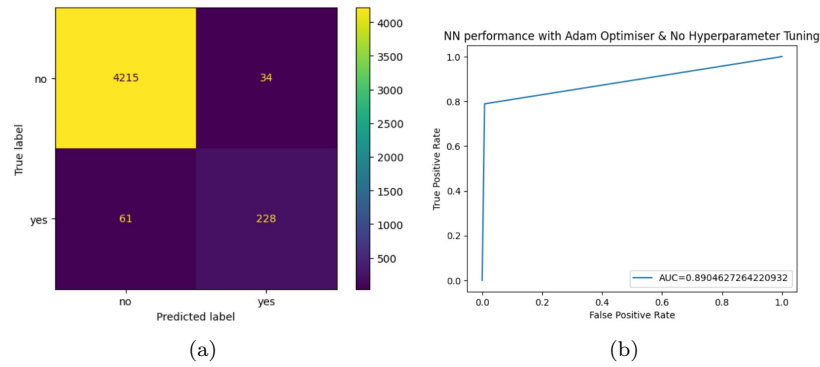


Figure 14: Model 27 with additional features - Adam optimiser: (a) Confusion Matrix; (b) ROC_AUC_Curve and AUC score.

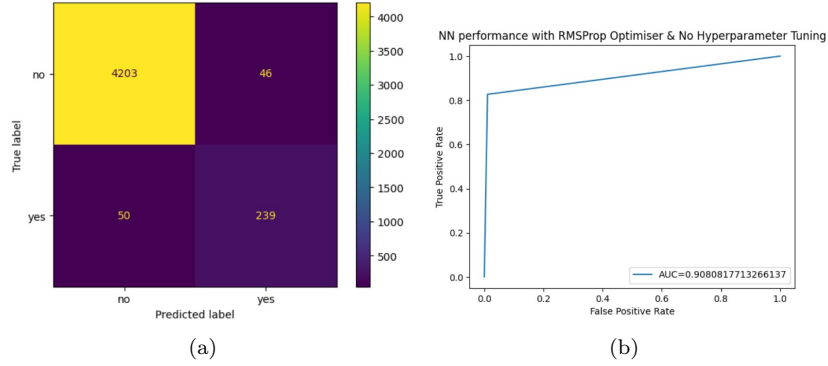


Figure 15: Model 28 with additional features - **Adam** optimiser: (a) Confusion Matrix; (b) ROC_AUC_Curve and AUC score.

3.2.2 NN Models with Hyperparameter Tuning

Models 29 to 40 for Adam optimiser and Models 41 to 52 for the RMSProp optimiser were trained, each for the same 12 different configurations of hyperparametric values as in Table 4, , with performace metrics for both of these groups in Tables 9 and 10.

With training and validation accuracy and loss curves in Figures 18 and 19, model 37 was found to be the best among all the neural network models, with an accuracy of about 98%, precision of 84% and recall of 78% (see Figure 20 for the confusion matrix and ROC curve).

Model	Optimiser	Accuracy	Precision	Recall	F1 Score
Model 29	Adam	0.974438	0.850202	0.726644	0.783582
Model 30	Adam	0.936316	0.000000	0.000000	0.000000
Model 31	Adam	0.936316	0.000000	0.000000	0.000000
Model 32	Adam	0.976642	0.823322	0.806228	0.814685
Model 33	Adam	0.977303	0.832143	0.806228	0.818981
Model 34	Adam	0.975760	0.840304	0.764706	0.800725
Model 35	Adam	0.979286	0.859779	0.806228	0.832143
Model 36	Adam	0.977303	0.822917	0.820069	0.821490
Model 37	Adam	0.976642	0.840149	0.782007	0.810036
Model 38	Adam	0.977082	0.813559	0.830450	0.821918
Model 39	Adam	0.977303	0.836957	0.799308	0.817699
Model 40	Adam	0.973116	0.819923	0.740484	0.778182

Table 9: Performance metrics for Adam optimiser-based models with additional features.

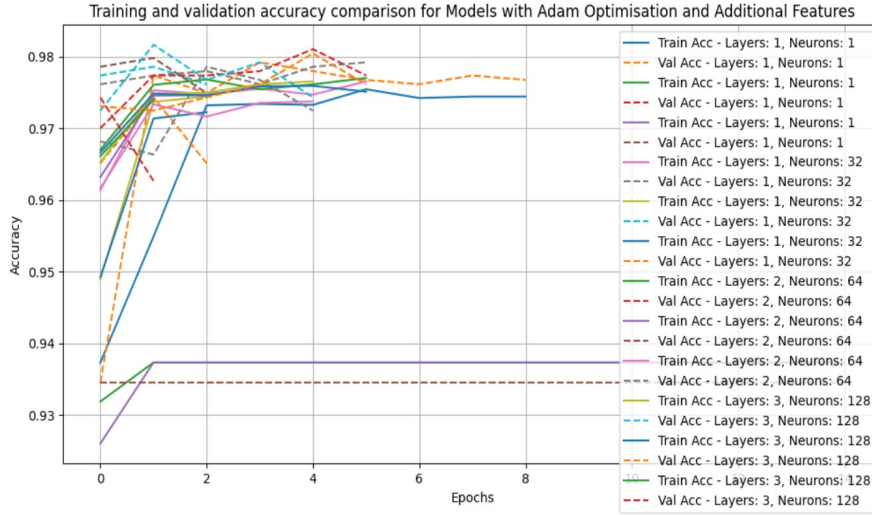


Figure 16: Training and Validation accuracy curves for various models with Adam Optimiser and hyperparametric tuning with additional features.

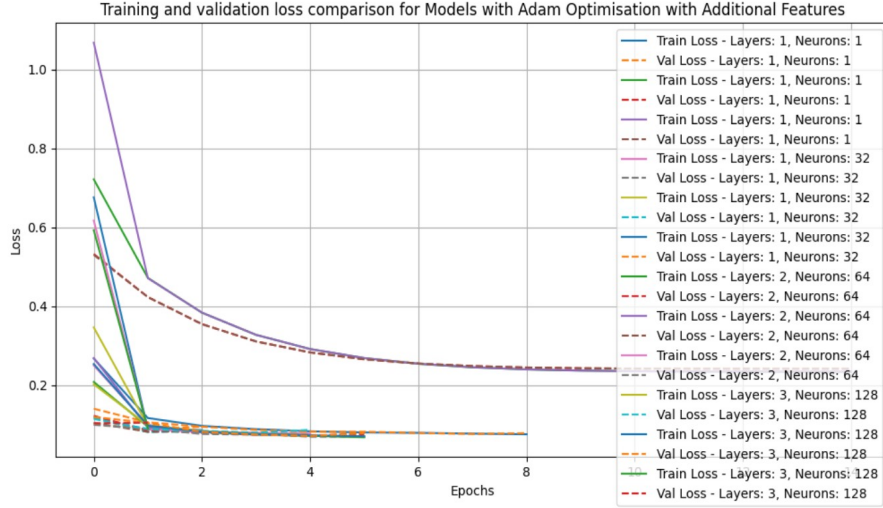


Figure 17: Training and Validation loss curves for various models with Adam Optimiser and hyperparametric tuning with additional features.

Model	Optimiser	Accuracy	Precision	Recall	F1 Score
Model 41	RMSPProp	0.976201	0.789137	0.854671	0.820598
Model 42	RMSPProp	0.975760	0.791531	0.840830	0.815436
Model 43	RMSPProp	0.971133	0.806202	0.719723	0.760512
Model 44	RMSPProp	0.975540	0.794702	0.830450	0.812183
Model 45	RMSPProp	0.979066	0.841549	0.826990	0.834206
Model 46	RMSPProp	0.973336	0.838710	0.719723	0.774674
Model 47	RMSPProp	0.977743	0.828671	0.820069	0.824348
Model 48	RMSPProp	0.975320	0.810526	0.799308	0.804878
Model 49	RMSPProp	0.973557	0.850622	0.709343	0.773585
Model 50	RMSPProp	0.977523	0.816949	0.833910	0.825342
Model 51	RMSPProp	0.976201	0.819788	0.802768	0.811189
Model 52	RMSPProp	0.975320	0.867220	0.723183	0.788679

Table 10: Performance metrics for RMSPProp optimiser-based models with additional features.

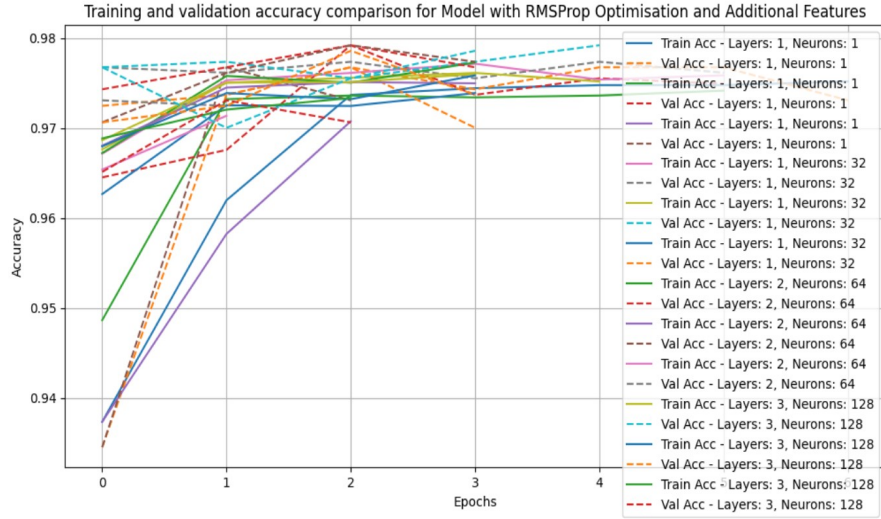


Figure 18: Training and Validation accuracy curves for various models with RMSProp Optimiser and hyperparametric tuning with additional features.

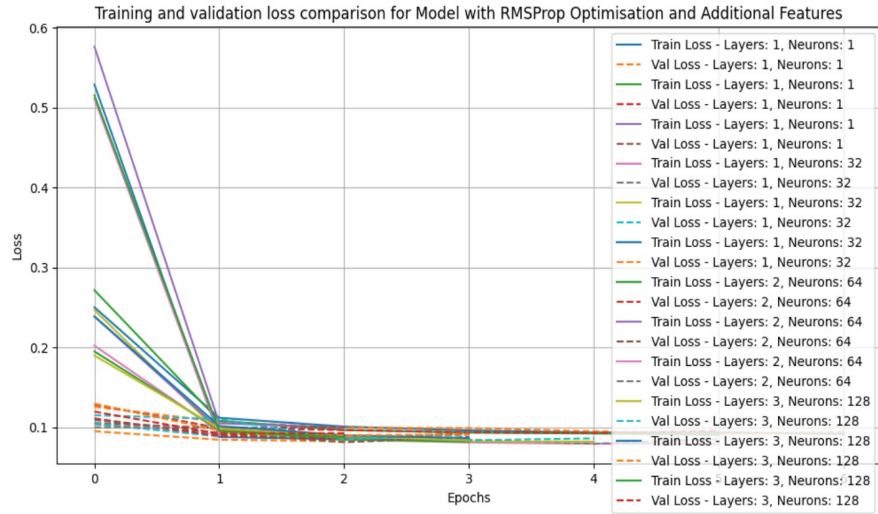


Figure 19: Training and Validation loss curves for various models with RMSProp Optimiser and hyperparametric tuning with additional features.

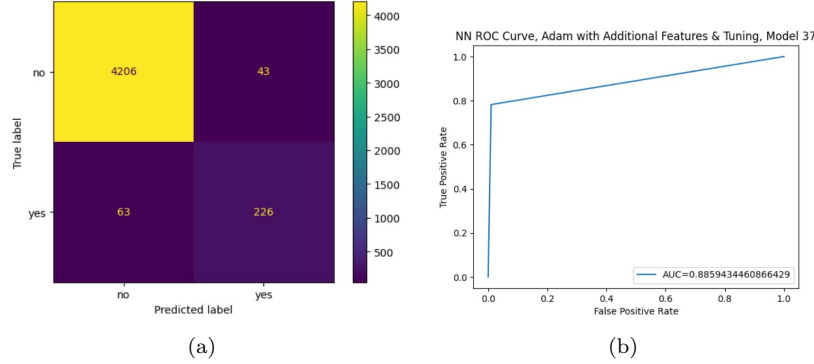


Figure 20: Model 37 with additional features - Adam optimiser: (a) Confusion Matrix; (b) ROC_AUC_Curve and AUC score.

Model Name	Accuracy	Precision	Recall	F1 Score
XGB_ORG	0.9689	0.78	0.72	0.75

Table 11: Result of training data with XGBoost model parameters WITHOUT hyperparametric tuning.

4 Modelling with XGBoost Algorithm

4.1 Modelling the Original Data with Dropped Features

4.1.1 XGBoost without Hyperparameter Tuning

The accuracy of XGBoost model XGB_ORG, without tuning (Table 11), is about 97%.. This is due to a very high number of true negatives and true positives (Figure 21(a)). True positives are large in proportion out of true and false positives taken together, hence a precision of 78%. However, recall is only 72% because of the higher number of false negatives. This model has the AUC score of 0.85 (Figure 21(b)).

4.1.2 XGBoost with Hyperparameter Tuning

Performance metrics of 27 XGBoost models, with varying combinations of hyperparameters, namely `learning_rate`, `n_estimators` and `max_depth`, were collected in Table 12. Three models, namely: XGB4, XGB7 and XGB10 (all with value of parameter `learning_rate` = 0.1) reported the highest accuracy of 97.38%, with XGB4 as the best model because of relatively higher precision 82% and relatively lower recall of 75%. But it was only marginally better in accuracy than XGB_ORG.

Model Name	n_est	max depth	learning rate	Acc	Precision	Recall	F1 Score
XGB1	100	3	0.1	0.9751	0.83	0.76	0.80
XGB2	100	3	0.25	0.9733	0.80	0.77	0.79
XGB3	100	3	0.5	0.9714	0.79	0.75	0.77
XGB4	100	4	0.1	0.9738	0.82	0.75	0.78
XGB5	100	4	0.25	0.9727	0.80	0.75	0.78
XGB6	100	4	0.5	0.9694	0.78	0.73	0.75
XGB7	100	5	0.1	0.9738	0.82	0.76	0.79
XGB8	100	5	0.25	0.9700	0.79	0.72	0.75
XGB9	100	5	0.5	0.9685	0.77	0.72	0.74
XGB10	200	3	0.1	0.9738	0.81	0.76	0.79
XGB11	200	3	0.25	0.9707	0.79	0.73	0.76
XGB12	200	3	0.5	0.9687	0.77	0.72	0.75
XGB13	200	4	0.1	0.9729	0.81	0.75	0.78
XGB14	200	4	0.25	0.9703	0.78	0.74	0.76
XGB15	200	4	0.5	0.9685	0.76	0.73	0.75
XGB16	200	5	0.1	0.9707	0.79	0.73	0.76
XGB17	200	5	0.25	0.9683	0.76	0.73	0.74
XGB18	200	5	0.5	0.9667	0.76	0.70	0.73
XGB19	300	3	0.1	0.9718	0.80	0.75	0.77
XGB20	300	3	0.25	0.9694	0.78	0.72	0.75
XGB21	300	3	0.5	0.9683	0.76	0.73	0.74
XGB22	300	4	0.1	0.9720	0.80	0.75	0.77
XGB23	300	4	0.25	0.9687	0.77	0.72	0.75
XGB24	300	4	0.5	0.9680	0.76	0.73	0.74
XGB25	300	5	0.1	0.9703	0.79	0.73	0.76
XGB26	300	5	0.25	0.9689	0.77	0.74	0.75
XGB27	300	5	0.5	0.9658	0.75	0.70	0.72

Table 12: Result of training data with XGBoost model parameters with hyperparametric tuning.

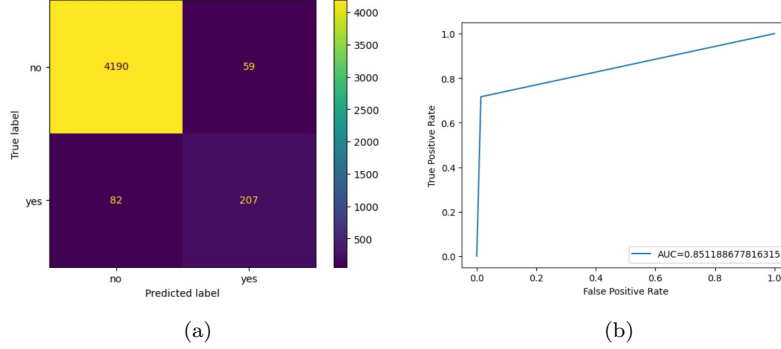


Figure 21: XGB_ORG Model: (a) Confusion Matrix; (b) ROC_AUC_Curve and AUC score.

Model Name	Accuracy	Precision	Recall	F1 Score
XGB_ADD	0.9815	0.87	0.83	0.85

Table 13: Results of training data (with additional features) with XGBoost model WITHOUT hyperparametric tuning.

4.2 Modelling Data with Additional Features

4.2.1 XGBoost without Hyperparameter Tuning

With the additional feature in XGBoost model XGB_ADD, visible improvement was noticed upon the models without these features (results in Table 13).

4.2.2 XGBoost with Hyperparameter Tuning

Another set of 27 models with varying combinations of hyperparametric tuning were defined and trained (results in Table 14), from which XGB46 gave a slight improvement over the XGB_ADD model (without tuning). The XGB46 model produced an accuracy of 98.26% with precision of 87% and recall of 85%. However, XGB_ADD had a low recall of 83% with a 98.15% accuracy which is not considerably less than XGB46 (Figure 22). Therefore, we regard XGB_ADD model as the best XGBoost model with additional features included.

4.2.3 Feature Importance

Feature importance for the XGB_ADD model has been shown through horizontal bar graph (Figure 23) and beeswarm plot (Figure 24) and the violin plot (Figure 25) using SHAP values.

Model Name	n_est	max depth	learning rate	Acc	Precision	Recall	F1 Score
XGB28	100	3	0.1	0.9799	0.86	0.82	0.84
XGB29	100	3	0.25	0.9813	0.86	0.84	0.85
XGB30	100	3	0.5	0.9802	0.85	0.83	0.84
XGB31	100	4	0.1	0.9808	0.86	0.84	0.85
XGB32	100	4	0.25	0.9819	0.87	0.84	0.86
XGB33	100	4	0.5	0.9802	0.87	0.81	0.84
XGB34	100	5	0.1	0.9806	0.85	0.84	0.85
XGB35	100	5	0.25	0.9788	0.85	0.82	0.83
XGB36	100	5	0.5	0.9799	0.86	0.81	0.84
XGB37	200	3	0.1	0.9817	0.86	0.85	0.86
XGB38	200	3	0.25	0.9808	0.86	0.84	0.85
XGB39	200	3	0.5	0.9795	0.85	0.82	0.84
XGB40	200	4	0.1	0.9822	0.87	0.85	0.86
XGB41	200	4	0.25	0.9799	0.85	0.83	0.84
XGB42	200	4	0.5	0.9802	0.86	0.83	0.84
XGB43	200	5	0.1	0.9804	0.85	0.84	0.84
XGB44	200	5	0.25	0.9797	0.86	0.82	0.84
XGB45	200	5	0.5	0.9795	0.86	0.82	0.84
XGB46	300	3	0.1	0.9826	0.87	0.85	0.86
XGB47	300	3	0.25	0.9802	0.85	0.83	0.84
XGB48	300	3	0.5	0.9804	0.85	0.84	0.84
XGB49	300	4	0.1	0.9819	0.87	0.84	0.86
XGB50	300	4	0.25	0.9808	0.86	0.83	0.85
XGB51	300	4	0.5	0.9799	0.86	0.82	0.84
XGB52	300	5	0.1	0.9802	0.86	0.82	0.84
XGB53	300	5	0.25	0.9799	0.85	0.83	0.84
XGB54	300	5	0.5	0.9797	0.86	0.82	0.84

Table 14: Result of training data with additional features and XGBoost model parameters with hyperparametric tuning

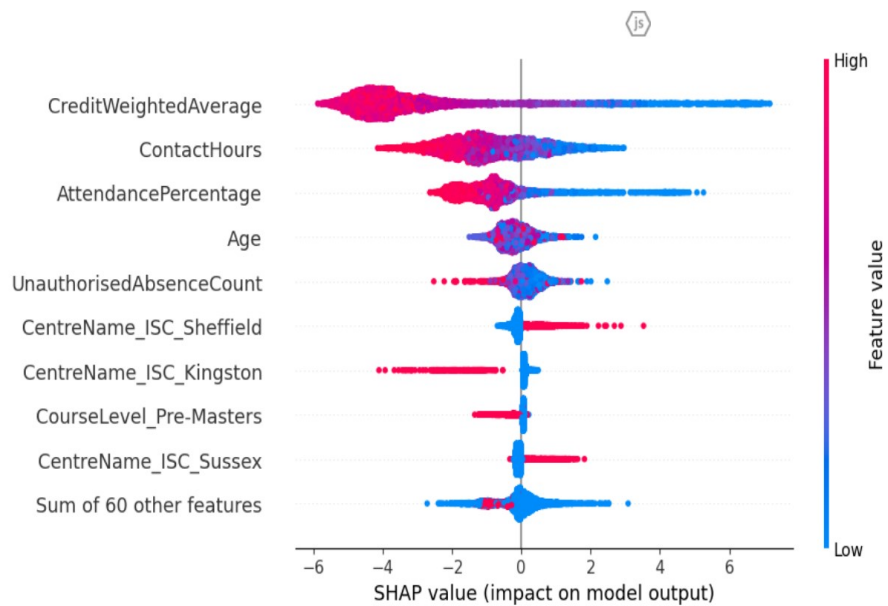


Figure 24: Feature Importance: Beeswarm chart for XGBoost model XGB_ADD with additional features.

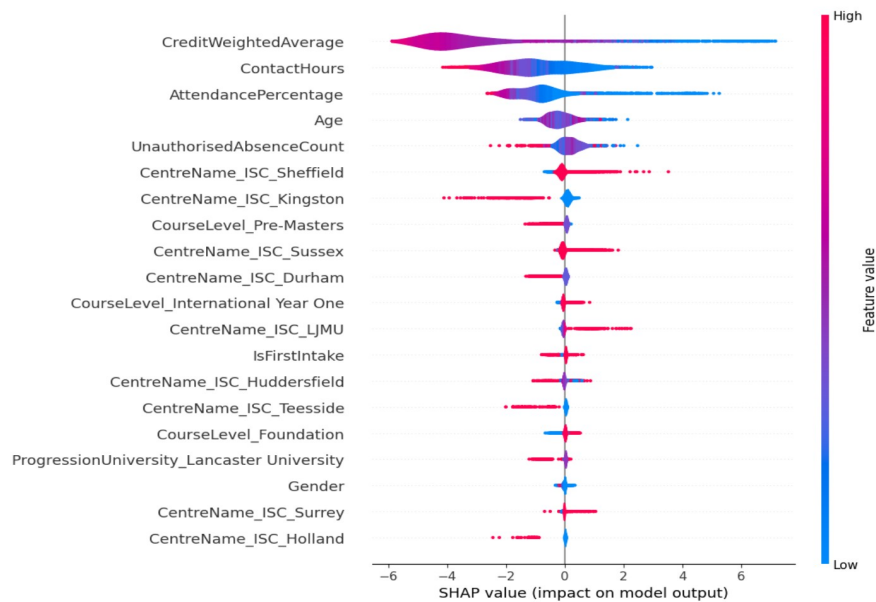


Figure 25: Feature Importance: Beeswarm chart for XGBoost model XGB_ADD with additional features.

5 Conclusions

Performance metrics for all the NN and XGBoost models showed, in general, improvement by including additional features over the models not having them. Hyperparametric tuning showed a minor improvement over models with no tuning, yet we consider that it was vital for the missing features to be included. This is because the student drop-out story will not be complete without including the **ContactHours** and **AttendancePercentage** features. This highlights the need for a critical analysis of all features before being dropped, especially when the dropped features were numerical. Feature importance plots in Figures 24 and 25 depict three most important features, two of which are these two features while also having a significant correlation with student **DropOut** variable.